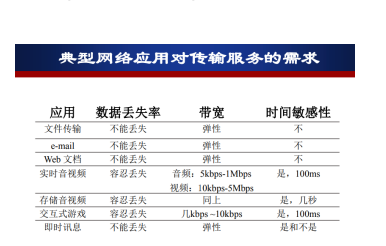
# 应用层

## 网络应用体系结构

- 客户/服务器结构(Client-Server, C/S)
  - 服务器的性能好坏决定了整个系统的性能
  - 注意：服务器在连接建立后也可以给客户主动发送数据
- 点对点结构(Peer-to-peer, P2P)
  - 优点:高度可伸缩
  - 缺点:难于管理
- 混合结构(Hybrid)

## 网络应用通信原理

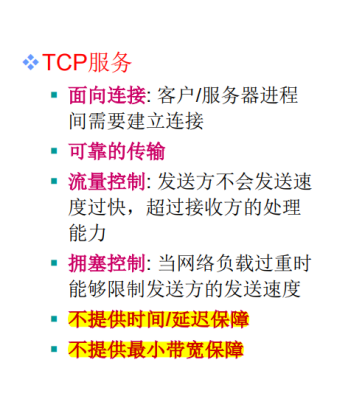
- 进程间通信
- 套接字: Socket — 进程间通信利用socket发送/接收消息实现
- 如何寻址进程? — IP地址+端口号
- 应用层协议
- 网络应用对传输服务的需求
  - 数据丢失(data loss)/可靠性(reliability)
  - 时间(timing)/延迟(delay)
  - 带宽(bandwidth)



典型网络应用对传输服务的需求

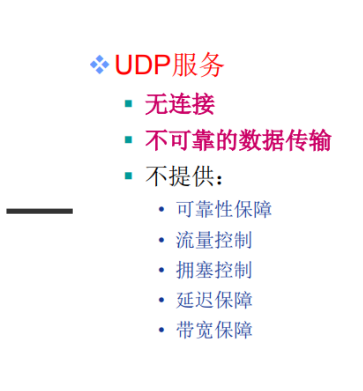
| 应用 | 数据丢失率 | 带宽 | 时间敏感性 |
|---|---|---|---|
| 文件传输 | 不能丢失 | 弹性 | 不 |
| e-mail | 不能丢失 | 弹性 | 不 |
| Web 文档 | 不能丢失 | 弹性 | 不 |
| 实时音视频 | 容忍丢失 | 音频：5kbps-1Mbps<br>视频：10kbps-5Mbps | 是，100ms |
| 存储音视频 | 容忍丢失 | 同上 | 是，几秒 |
| 交互式游戏 | 容忍丢失 | 几kbps –10kbps | 是，100ms |
| 即时讯息 | 容忍丢失 | 弹性 | 是和不是 |

- Internet提供的传输服务
  - TCP服务



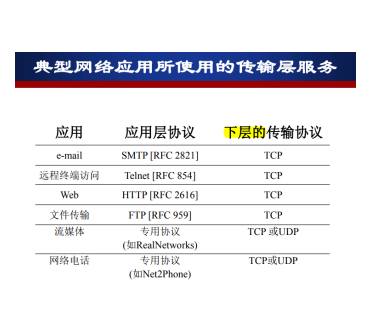
❖TCP服务
- 面向连接: 客户/服务器进程间需要建立连接
- 可靠的传输
- 流量控制: 发送方不会发送速度过快，超过接收方的处理能力
- 拥塞控制: 当网络负载过重时能够限制发送方的发送速度
- 不提供时间/延迟保障
- 不提供最小带宽保障

  - UDP服务



❖UDP服务
- 无连接
- 不可靠的数据传输
- 不提供:
  - 可靠性保障
  - 流量控制
  - 拥塞控制
  - 延迟保障
  - 带宽保障

  - 举例

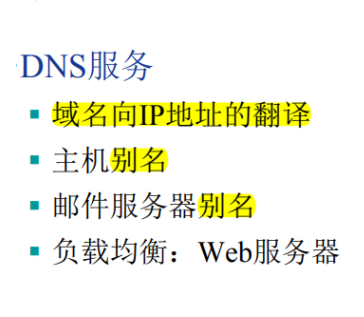


典型网络应用所使用的传输层服务

| 应用 | 应用层协议 | 下层的传输协议 |
|---|---|---|
| e-mail | SMTP [RFC 2821] | TCP |
| 远程终端访问 | Telnet [RFC 854] | TCP |
| Web | HTTP [RFC 2616] | TCP |
| 文件传输 | FTP [RFC 959] | TCP |
| 流媒体 | 专用协议<br>(如RealNetworks) | TCP 或UDP |
| 网络电话 | 专用协议<br>(如Net2Phone) | TCP或UDP |

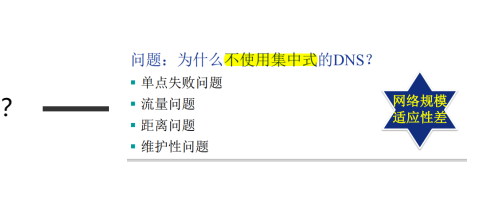
## 域名解析系统(DNS)

- 域名和IP地址之间的映射问题
- DNS服务



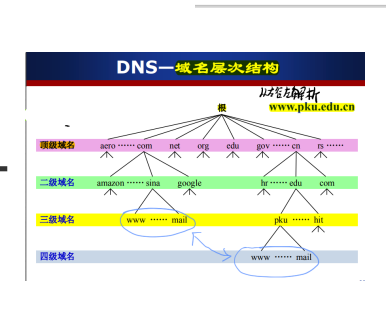
DNS服务
- 域名向IP地址的翻译
- 主机别名
- 邮件服务器别名
- 负载均衡：Web服务器

- 为什么不使用集中式的DNS?



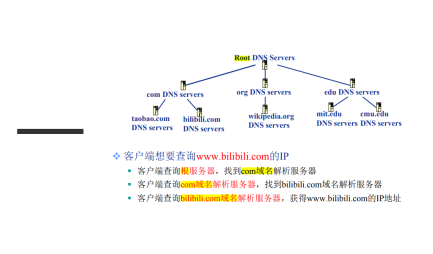
问题：为什么不使用集中式的DNS?
- 单点失败问题
- 流量问题
- 距离问题
- 维护性问题

- 域名层次结构

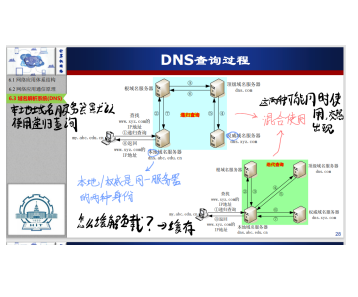


DNS—域名层次结构
www.pku.edu.cn
根
顶级域名
二级域名
三级域名
四级域名

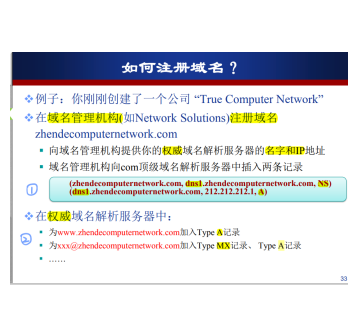
- 分布式层次式数据库



- 解析服务器类型
  - 本地域名解析服务器
    - 避免每次都要访问根域名服务器
    - 不严格属于层级体系
    - 当主机进行DNS查询时，查询首先被发送到本地域名服务器 — 作为代理(proxy)，将查询转发给（层级式）域名解析服务器系统
  - DNS根域名服务器
    - 本地域名解析服务器无法解析域名时，访问根域名服务器
    - 如果不知道映射，则查询顶级域名服务器，……，查询权威域名服务器
  - 顶级域名服务器 — 负责com, org, net, edu等顶级域名和国家顶级域名，例如cn, uk, fr等
  - 权威(Authoritative)域名服务器（王道：权限域名服务器）
    - 组织的域名解析服务器，供组织内部服务器的解析服务
    - 到达权威域名服务器后即可获取所有映射(终点)
    - 王道：将其管辖的主机名转化为IP地址，每台主机都必须在权限域名服务器处登记
    - 事实上，许多域名服务器同时充当本地域名服务器和权威域名服务器
- DNS查询过程
  - 递归查询 — 主机对本地域名服务器的访问采用递归查询
  - 迭代查询

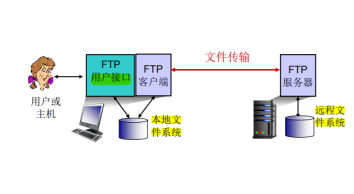


- DNS记录缓存和更新
  - 只要域名解析服务器获得域名—IP映射，即缓存这一映射
    - 本地域名服务器一般会缓存顶级域名服务器的映射（因为顶级域名服务器太常见），因此根域名服务器不经常被访问
    - 一段时间过后,缓存条目会失效(删除)
  - DNS记录
    - 资源记录 **RR format: (name, value, type, ttl)**
    - Type
      - A
        - A就是Address的首字母，这种记录就是将域名映射到一个IPv4地址上，这也是最常用的DNS记录类型
        - Name: 主机域名 Value: IP地址 — Canonical name record
      - NS
        - Name server record 用于存储DNS记录并将其信息提供给需要查询的客户端，NS记录将会为DNS服务指定名称和IP地址，一条NS记录可以包含多个DNS服务器
        - Name: 域(edu.cn) Value: 该域权威域名解析服务器的主机域名
      - CNAME
        - Canonical name record别名记录，一个常用的例子是将知域名指向www域名。比如：example.com IN CNAME www.example.com
        - Name: 某一真实域名的别名 Value: 真实域名
      - MX
        - Mail exchange record 用于email邮箱地址解析
        - Name: 邮件服务器别名 Value: 邮件服务器名
- 如何注册域名

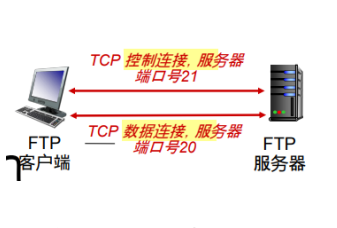


## FTP应用

- FTP文件传输协议

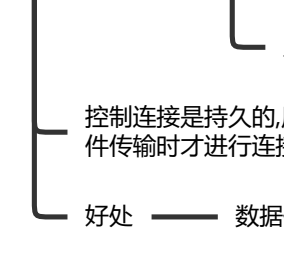


  - 有状态协议 — FTP服务器维护用户的状态信息：当前路径、用户帐户与控制连接对应
  - 带外连接：控制连接与数据连接分开



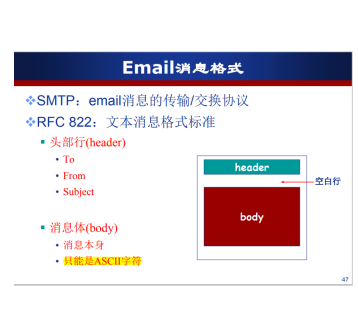
TCP 控制连接 端口号21
TCP 数据连接 端口号20
FTP 客户端
FTP 服务器

    - FTP连接使用TCP作为传输协议
    - 控制连接
      - 使用端口21作为联系
      - 客户端通过控制连接获得身份确认
      - 客户端通过控制连接发送命令浏览远程目录
      - 收到一个文件传输命令时，服务器打开一个到客户端的TCP数据连接
    - 数据连接
      - 使用端口20
      - 一个文件传输完成后，服务器关闭连接
      - 要想传输另外一个文件，则要再打开第二个TCP数据连接
      - 王道：主动模式PORT和被动模式PASV — 默认主动模式PORT
    - 控制连接是持久的,用户上线后便一直连接,直到下线,数据连接是暂时的,只有在文件传输时才进行连接
    - 好处 — 数据传输不断用户与服务器进行交互
  - FTP 命令、响应

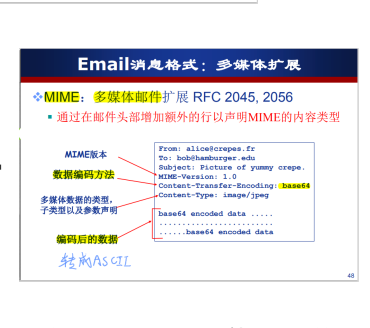


## Email应用

- 王道：一个电子邮件系统的构成组件
  - 用户代理
  - 邮件服务器
  - 电子邮件使用的协议
- Email消息格式
  - 王道：构成
    - 信封
    - 内容
      - 首部 — 关键词：To、From、Subject
      - 主体



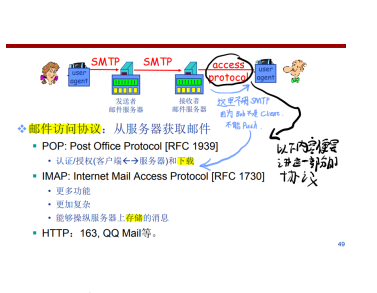
- MIME：多媒体邮件扩展



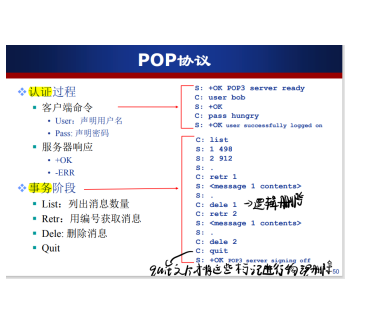
Email消息格式：多媒体扩展

  - 王道：由于SMTP只能传送7位ASCII码文本邮件，无法传送可执行文件以及其他二进制对象，因此提出了MIME，对于非ASCII码数据，MIME会将其转换为ASCII码数据，然后使用SMTP进行传送
- SMTP协议
  - 使用TCP进行email消息的可靠传输，端口号为25
  - 命令/响应交互模式
    - 命令(command): ASCII文本
    - 响应(response): 状态代码和语句
  - Email消息只能包含7位ASCII码
  - 使用持久性连接
  - 传输过程的三个阶段
    - 握手
    - 消息的传输
    - 关闭
  - 连接建立后的交互示例

    - 对着王道看看
  - 对比HTTP协议
    - 相同
      - 都使用命令/响应交互模式
      - 命令和状态代码都是ASCII码
    - 区别
      - HTTP: 拉式(pull)-->主动下载<br>SMTP:推式(push)-->主动上传
      - HTTP: 每个对象封装在独立的响应消息中<br>SMTP: 多个对象在由多个部分构成的消息中发送
- 邮件访问协议
  - 从服务器获取邮件



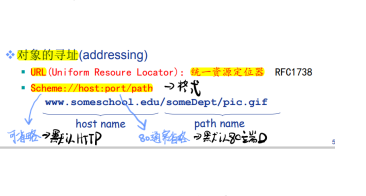
  - POP协议
    - 使用TCP连接，端口号110
    - 工作



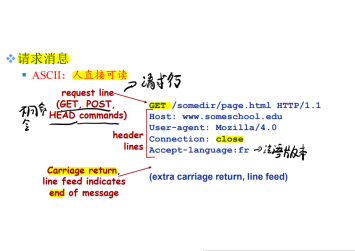
    - 模式
      - "下载并删除"模式
        - 节省空间
        - 用户如果换了客户端软件，无法重读该邮件
      - "下载并保持"模式 — 不同客户端都可以保留消息的拷贝
    - POP3是无状态的
  - IMAP协议
    - 所有消息统一保存在一个地方：服务器
    - 允许用户利用文件夹组织消息
    - IMAP支持跨会话(Session)的用户状态
      - 文件夹的名字
      - 文件夹与消息ID之间的映射等

## Web应用

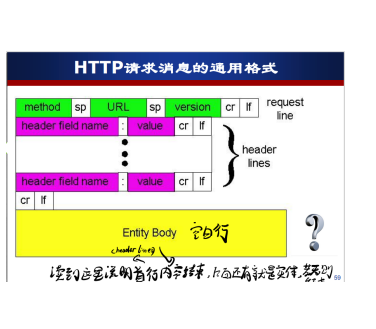
- 网页(Web Page)包含多个对象
  - 对象：HTML文件、JPEG图片、视频文件、动态脚本等
  - 基本HTML文件：包含对其他对象引用的链接
- 王道：万维网WWW的内核部分
  - 统一资源定位符URL — 每个文档在整个万维网的范围内具有唯一的标识符URL — URL的一般形式：<协议>://<主机>:<端口>/<路径>（端口和路径有时候可以省略）
  - 超文本传输协议HTTP
  - 超文本标记语言HTML
- 对象的寻址(addressing)



- HTTP超文本传输协议
  - 概述
    - C/S 客户端/服务器结构
    - 使用TCP传输服务 — 服务器在80端口等待客户的请求
    - 无状态协议 — 可以使用cookie
    - HTTP的报文结构
      - 两类消息
        - 请求消息

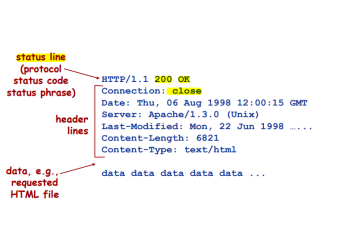


          - 通用格式



HTTP请求消息的通用格式
method
URL
version
header field name
value
header lines
request line
Entity Body

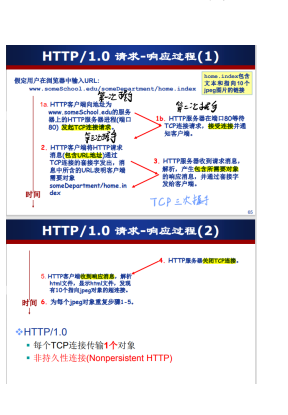
          - 上传输入的方法
            - POST方法
              - 网页经常需要填写表格
              - 在请求消息的消息体(entity body)中上传客户端的输入
            - URL方法
              - 使用GET方法
              - 输入信息通过request行的URL字段上传
        - 响应消息



          - 响应状态代码

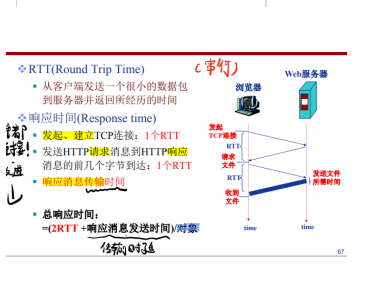
HTTP响应状态代码
404 Not Found
200 OK
301 Moved Permanently
400 Bad Request
404 Not Found
505 HTTP Version Not Supported

      - 王道：HTTP请求报文的常用的几个方法
        - GET — 请求读取由URL标识的信息
        - HEAD — 使用HEAD方法，服务器可对HTTP报文进行响应，但不会返回请求对象，其主要作用是调试
        - POST — 给服务器添加信息（如注释）
        - CONNECT — 用于代理服务器
  - HTTP版本
    - HTTP/1.0
      - 概述
        - 每个TCP连接传输一个对象
        - 非持久性连接 ->连接完成后断开
      - 请求-响应过程

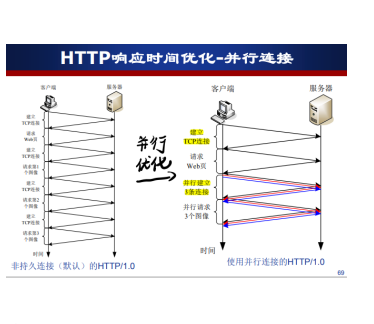


HTTP/1.0 请求-响应过程(1)
HTTP/1.0 请求-响应过程(2)

      - 响应时间分析

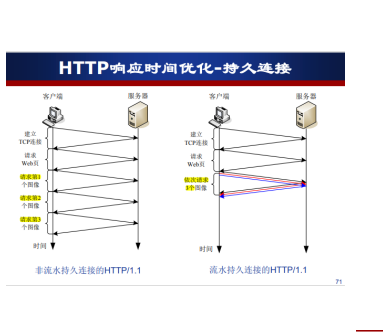


      - 缺点 — 每传输一个文件都得重新创建连接,麻烦!
      - 响应时间优化
        - 并行连接



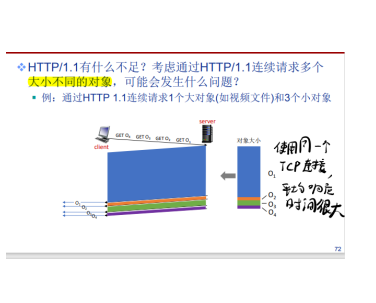
HTTP响应时间优化-并行连接

        - 持久连接
          - 发送响应后，服务器保持TCP连接打开
          - 后续HTTP消息可以通过这个连接发送
          - 每个TCP连接允许传输多个对象
        - 流水 — 客户端只要遇到一个引用对象就尽快发出请求
    - HTTP/1.1
      - 使用最多的版本
      - 默认采用流水的持久连接 — 理想情况下，收到所有的引用对象只需耗时约1个RTT

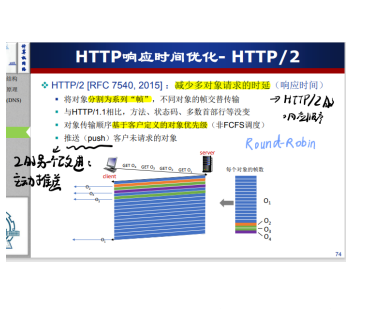


HTTP响应时间优化-持久连接

      - 不足
        - 使用同一个TCP连接，使用FCFS调度，在大对象之后请求的小对象，需要等待大对象的传输，请求文件的平均响应时间很大

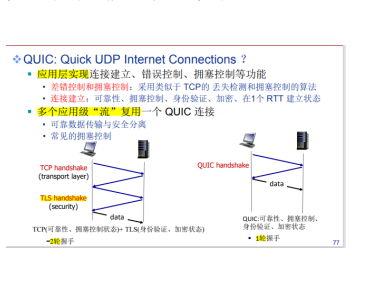


        - 若发生数据丢失会暂停对象传输
    - HTTP/2
      - 将对象分割为系列"帧"，不同对象的帧交替传输，减少小对象请求的时延
      - 主动推送 — 推送（push）客户未请求的对象



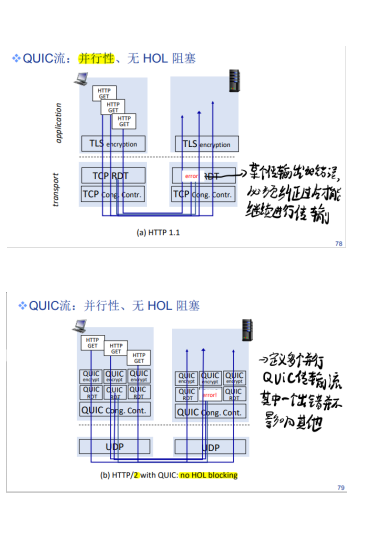
HTTP响应时间优化- HTTP/2

      - 不足 — 使用单一的TCP连接，丢包恢复仍然会暂停对象传输
    - HTTP/3
      - 在 UDP 之上实现应用，将传输层功能移到应用层 — 前面的HTTP协议都是TCP协议基础上的, 而HTTP/3是UDP协议基础
      - 使用QUIC协议，在UDP之上重新设计了一个协议
      - 应用层实现连接建立、错误控制、拥塞控制等功能
        - 差错控制和拥塞控制：采用类似于 TCP的 丢失检测和拥塞控制的算法
        - 连接建立：可靠性、拥塞控制、身份验证、加密、在1个 RTT 建立状态
      - 多个应用级"流"复用一个 QUIC 连接



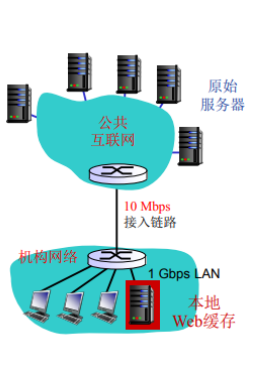
QUIC: Quick UDP Internet Connections

      - QUIC流：并行性、无 HOL 阻塞

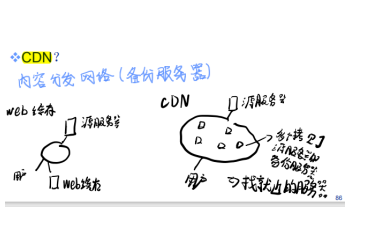


QUIC流：并行性、无 HOL 阻塞

      - 未来可期的版本
  - Web缓存/代理服务器
    - 进一步优化HTTP响应时间
    - 好处
      - 缓存更靠近客户端，减少客户端请求的响应时间
      - 减少机构访问链路上的流量
      - 互联网密集部署缓存

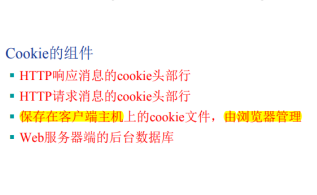


  - 其他优化Web应用响应时间的方案 — CDN内容分发网络（备份服务器）

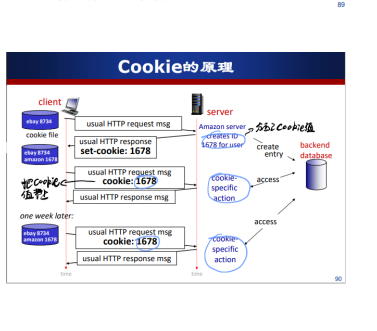


  - 条件性GET方法

  - Cookie
    - 无状态HTTP协议的优缺点
      - 好处：容易维护
      - 坏处：无法掌握状态
    - Cookie：某些网站为了辨别用户身份，进行session跟踪而储存在用户本地终端上的数据（通常经过加密）
    - 组件



    - 原理

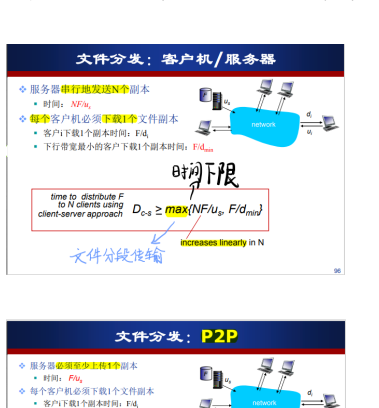


    - 存在有隐私问题
- HTTP与SMTP的比较

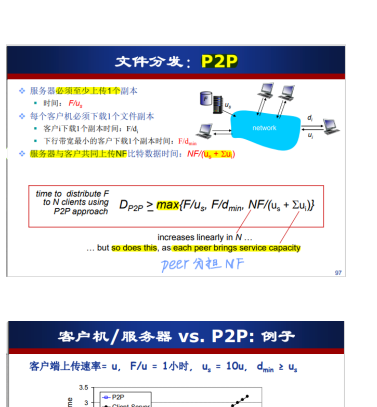
HTTP vs SMTP

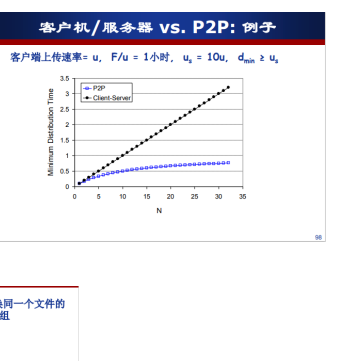
## P2P应用

- 没有服务器,任意端系统之间直接通信
- 文件分发：客户机/服务器 vs. P2P
  - 问题: 从一个服务器向N个节点分发一个文件需要多长时间

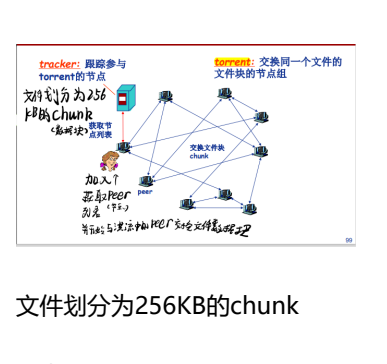


文件分发：客户机/服务器

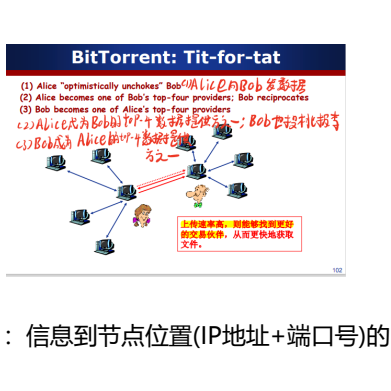




- BitTorrent

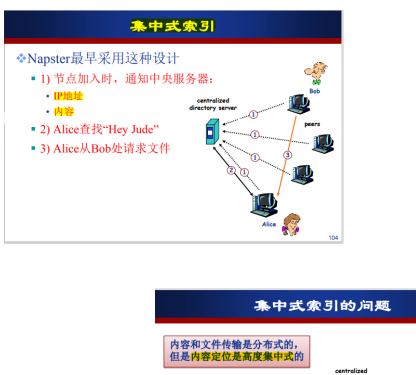


  - 文件划分为256KB的chunk
  - 节点加入torrent
  - 下载的同时，节点需要向其他节点上传chunk
  - 获取chunk
    - 给定任一时刻，不同的节点持有文件的不同chunk集合
    - 节点(Alice)定期查询每个邻居所持有的chunk列表
    - 节点发送请求，请求获取缺失的chunk -->稀缺优先 — 指获取那些份数少的chunk，这使得能提供这个chunk的peer变多，平衡洪流中的chunk
  - 发送chunk
    - tit-for-tat 互惠
    - 结点Alice向4个邻居发送chunk：正在向其发送Chunk，速率最快的4个(最好的朋友, 礼尚往来) — 每10秒重新评估top 4
    - 每30秒随机选择一个其他节点，向其发送chunk — 新选择节点可能会加入top 4
  - 示例

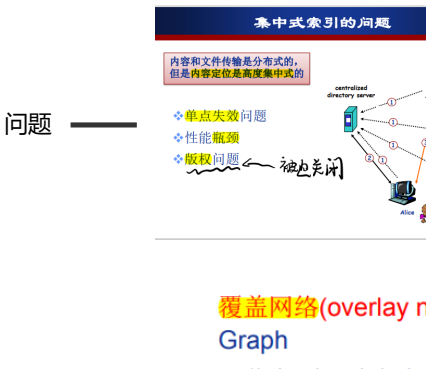


BitTorrent: Tit-for-tat

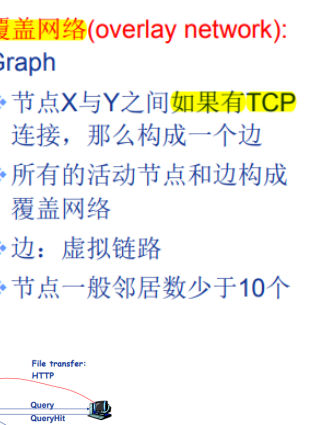
- 搜索信息
  - P2P系统的索引：信息到节点位置(IP地址+端口号)的映射
  - 类型
    - 集中式索引



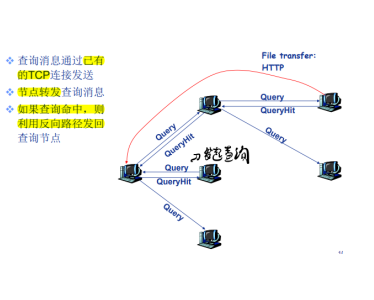
      - 问题



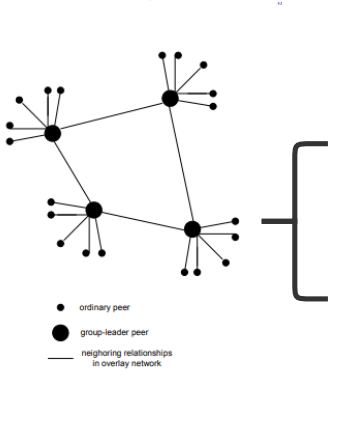
    - 洪泛式查询
      - 覆盖网络



覆盖网络(overlay network): Graph
- ❖节点X与Y之间如果有TCP连接，那么构成一个边
- ❖所有的活动节点和边构成覆盖网络
- ❖边：虚拟链路
- ❖节点一般邻居数少于10个

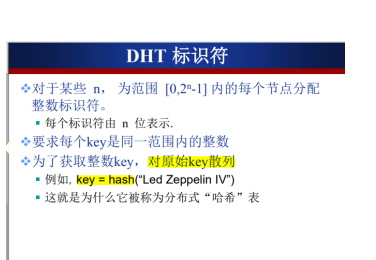


    - 层次式覆盖网络

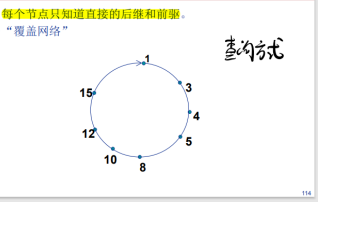


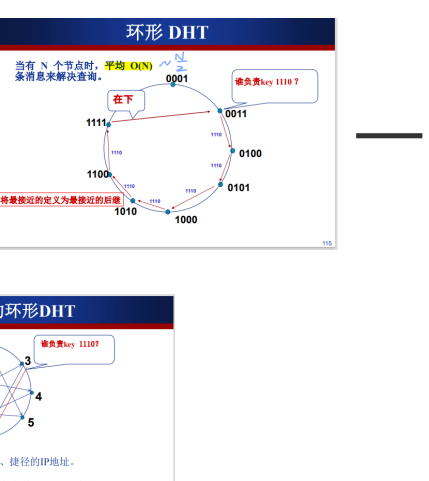
      - 每个节点或者是一个超级节点，或者被分配一个超级节点
        - 节点和超级节点间维持TCP连接
        - 某些超级节点对之间维持TCP连接
        - 超级节点负责跟踪子节点的内容
      - 介于集中式索引和洪泛式查询之间的方法
    - 分布式哈希表DHT
      - DHT: 一种分布式P2P数据库

      - 如何将key分配给节点?
        - 基本思想
          - 将每个key转换为整数
          - 为每个节点分配整数
          - 将（key，value）对放在最接近键的节点中
        - DHT 标识符



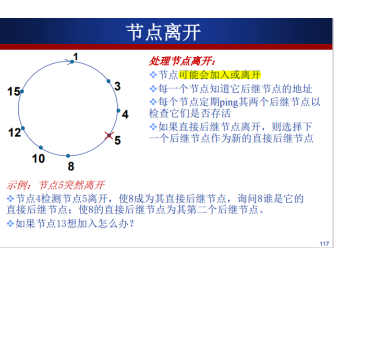
      - 环形 DHT
        - 每个节点只知道直接的后继和前驱





          - 查询时间O(N)
        - 带捷径的环形DHT

          - 查询时间O(log N)
      - 处理节点的加入与离开



## 一些总结

- 在应用层协议中, 哪些是用了TCP协议, 哪些是用了UDP协议?
  - TCP协议
    - HTTP — 默认端口为80
    - FTP — 20用于传输数据，21用于传输控制信息
    - SMTP — 25
    - POP — 110
    - 边界网关协议BGP
  - UDP协议
    - DNS — 53
    - HTTP/3 — 基于UDP之上, 使用QUIC协议
    - 路由信息协议RIP
    - DHCP协议

## 分支主题9